

NLTK 3.2.4 documentation

MODULES | INDEX

Source code for nltk.corpus.reader.wordnet

```
# -*- coding: utf-8 -*-
# Natural Language Toolkit: WordNet
#
# Copyright (C) 2001-2017 NLTK Project
# Author: Steven Bethard <Steven.Bethard@colorado.edu>
#         Steven Bird <stevenbird1@gmail.com>
#         Edward Loper <edloper@gmail.com>
#         Nitin Madnani <nmadnani@ets.org>
#         Nasruddin A'aidil Shari
#         Sim Wei Ying Geraldine
#         Soe Lynn
#         Francis Bond <bond@ieee.org>
# URL: <http://nltk.org/>
# For license information, see LICENSE.TXT

"""
An NLTK interface for WordNet

WordNet is a lexical database of English.
Using synsets, helps find conceptual relationships between words
such as hypernyms, hyponyms, synonyms, antonyms etc.

For details about WordNet see:
http://wordnet.princeton.edu/

This module also allows you to find lemmas in languages
other than English from the Open Multilingual Wordnet
http://compling.hss.ntu.edu.sg/omw/

"""

from __future__ import print_function, unicode_literals

import math
import re
from itertools import islice, chain
from functools import total_ordering
from operator import itemgetter
from collections import defaultdict, deque

from six import iteritems
from six.moves import range

from nltk.corpus.reader import CorpusReader
from nltk.util import binary_search_file as _binary_search_file
from nltk.probability import FreqDist
from nltk.compat import python_2_unicode_compatible
from nltk.internals import deprecated

#####
# Table of Contents
#####
# - Constants
# - Data Classes
#   - WordNetError
#   - Lemma
#   - Synset
# - WordNet Corpus Reader
# - WordNet Information Content Corpus Reader
# - Similarity Metrics
# - Demo

#####
```

TABLE OF CONTENTS

NLTK News
Installing NLTK
Installing NLTK Data
Contribute to NLTK
FAQ
Wiki
API
HOWTO

SEARCH

Go

```

# Constants
#####

#: Positive infinity (for similarity functions)
_INF = 1e300

# { Part-of-speech constants
ADJ, ADJ_SAT, ADV, NOUN, VERB = 'a', 's', 'r', 'n', 'v'
# }

POS_LIST = [NOUN, VERB, ADJ, ADV]

# A table of strings that are used to express verb frames.
VERB_FRAME_STRINGS = (
    None,
    "Something %s",
    "Somebody %s",
    "It is %sing",
    "Something is %sing PP",
    "Something %s something Adjective/Noun",
    "Something %s Adjective/Noun",
    "Somebody %s Adjective",
    "Somebody %s something",
    "Somebody %s somebody",
    "Something %s somebody",
    "Something %s something",
    "Something %s to somebody",
    "Somebody %s on something",
    "Somebody %s somebody something",
    "Somebody %s something to somebody",
    "Somebody %s something from somebody",
    "Somebody %s somebody with something",
    "Somebody %s somebody of something",
    "Somebody %s something on somebody",
    "Somebody %s somebody PP",
    "Somebody %s something PP",
    "Somebody %s PP",
    "Somebody's (body part) %s",
    "Somebody %s somebody to INFINITIVE",
    "Somebody %s somebody INFINITIVE",
    "Somebody %s that CLAUSE",
    "Somebody %s to somebody",
    "Somebody %s to INFINITIVE",
    "Somebody %s whether INFINITIVE",
    "Somebody %s somebody into V-ing something",
    "Somebody %s something with something",
    "Somebody %s INFINITIVE",
    "Somebody %s VERB-ing",
    "It %s that CLAUSE",
    "Something %s INFINITIVE")

SENSENUM_RE = re.compile(r'\.d\d\.')

#####
# Data Classes
#####

class WordNetError(Exception):
    """An exception class for wordnet-related errors."""

    @total_ordering
    class _WordNetObject(object):
        """A common base class for lemmas and synsets."""

        def hypernyms(self):
            return self._related('@')

```

[\[docs\]](#)

```
def _hypernyms(self):
    return self._related('@', sort=False)

def instance_hypernyms(self):
    return self._related('@i')

def _instance_hypernyms(self):
    return self._related('@i', sort=False)

def hyponyms(self):
    return self._related('~')

def instance_hyponyms(self):
    return self._related('~i')

def member_holonyms(self):
    return self._related('#m')

def substance_holonyms(self):
    return self._related('#s')

def part_holonyms(self):
    return self._related('#p')

def member_meronyms(self):
    return self._related('%m')

def substance_meronyms(self):
    return self._related('%s')

def part_meronyms(self):
    return self._related('%p')

def topic_domains(self):
    return self._related(';c')

def region_domains(self):
    return self._related(';r')

def usage_domains(self):
    return self._related(';u')

def attributes(self):
    return self._related('=')

def entailments(self):
    return self._related('*')

def causes(self):
    return self._related('>')

def also_sees(self):
    return self._related('^')

def verb_groups(self):
    return self._related('$')

def similar_tos(self):
    return self._related('&')

def __hash__(self):
    return hash(self._name)

def __eq__(self, other):
    return self._name == other._name

def __ne__(self, other):
    return self._name != other._name

def __lt__(self, other):
    return self._name < other._name
```

@python_2_unicode_compatible

class Lemma(_WordNetObject):

[\[docs\]](#)

"""

The lexical entry for a single morphological form of a sense-disambiguated word.

Create a Lemma from a "<word>.<pos>.<number>.<Lemma>" string where:

<word> is the morphological stem identifying the synset

<pos> is one of the module attributes ADJ, ADJ_SAT, ADV, NOUN or VERB

<number> is the sense number, counting from 0.

<Lemma> is the morphological form of interest

Note that <word> and <Lemma> can be different, e.g. the Synset

'salt.n.03' has the Lemmas 'salt.n.03.salt', 'salt.n.03.saltiness' and

'salt.n.03.salinity'.

Lemma attributes, accessible via methods with the same name::

- name: The canonical name of this Lemma.
- synset: The synset that this Lemma belongs to.
- syntactic_marker: For adjectives, the WordNet string identifying the syntactic position relative modified noun. See: <http://wordnet.princeton.edu/man/wninput.5WN.html#sect10> For all other parts of speech, this attribute is None.
- count: The frequency of this Lemma in wordnet.

Lemma methods:

Lemmas have the following methods for retrieving related Lemmas. They

correspond to the names for the pointer symbols defined here:

<http://wordnet.princeton.edu/man/wninput.5WN.html#sect3>

These methods all return Lists of Lemmas:

- antonyms
- hypernyms, instance_hypernyms
- hyponyms, instance_hyponyms
- member_holonyms, substance_holonyms, part_holonyms
- member_meronyms, substance_meronyms, part_meronyms
- topic_domains, region_domains, usage_domains
- attributes
- derivationally_related_forms
- entailments
- causes
- also_sees
- verb_groups
- similar_tos
- pertainyms

"""

```
__slots__ = ['_wordnet_corpus_reader', '_name', '_syntactic_marker',
             '_synset', '_frame_strings', '_frame_ids',
             '_lexname_index', '_lex_id', '_lang', '_key']
```

```
def __init__(self, wordnet_corpus_reader, synset, name,
             lexname_index, lex_id, syntactic_marker):
    self._wordnet_corpus_reader = wordnet_corpus_reader
    self._name = name
    self._syntactic_marker = syntactic_marker
    self._synset = synset
    self._frame_strings = []
    self._frame_ids = []
    self._lexname_index = lexname_index
    self._lex_id = lex_id
    self._lang = 'eng'

    self._key = None # gets set later.
```

```
def name(self):
    return self._name
```

[\[docs\]](#)

```

def syntactic_marker(self): \[docs\]
    return self._syntactic_marker

def synset(self): \[docs\]
    return self._synset

def frame_strings(self): \[docs\]
    return self._frame_strings

def frame_ids(self): \[docs\]
    return self._frame_ids

def lang(self): \[docs\]
    return self._lang

def key(self): \[docs\]
    return self._key

def __repr__(self):
    tup = type(self).__name__, self._synset._name, self._name
    return "%s('%s.%s')" % tup

def _related(self, relation_symbol):
    get_synset = self._wordnet_corpus_reader.synset_from_pos_and_offset
    return sorted([
        get_synset(pos, offset)._lemmas[lemma_index]
        for pos, offset, lemma_index
        in self._synset._lemma_pointers[self._name, relation_symbol]
    ])

def count(self): \[docs\]
    """Return the frequency count for this Lemma"""
    return self._wordnet_corpus_reader.lemma_count(self)

def antonyms(self): \[docs\]
    return self._related('!')

def derivationally_related_forms(self): \[docs\]
    return self._related('+')

def pertainyms(self): \[docs\]
    return self._related('\\')

```

@python_2_unicode_compatible

```
class Synset(_WordNetObject): \[docs\]
```

"""Create a Synset from a "<Lemma>.<pos>.<number>" string where:
 <Lemma> is the word's morphological stem
 <pos> is one of the module attributes ADJ, ADJ_SAT, ADV, NOUN or VERB
 <number> is the sense number, counting from 0.

Synset attributes, accessible via methods with the same name:

- name: The canonical name of this synset, formed using the first Lemma of this synset. Note that this may be different from the name passed to the constructor if that string used a different Lemma to identify the synset.
- pos: The synset's part of speech, matching one of the module level attributes ADJ, ADJ_SAT, ADV, NOUN or VERB.

- *lemmas*: A list of the *Lemma* objects for this synset.
- *definition*: The definition for this synset.
- *examples*: A list of example strings for this synset.
- *offset*: The offset in the WordNet dict file of this synset.
- *lexname*: The name of the Lexicographer file containing this synset.

Synset methods:

Synsets have the following methods for retrieving related Synsets. They correspond to the names for the pointer symbols defined here: <http://wordnet.princeton.edu/man/wninput.5WN.html#sect3> These methods all return lists of Synsets.

- *hypernyms*, *instance_hypernyms*
- *hyponyms*, *instance_hyponyms*
- *member_holonyms*, *substance_holonyms*, *part_holonyms*
- *member_meronyms*, *substance_meronyms*, *part_meronyms*
- *attributes*
- *entailments*
- *causes*
- *also_sees*
- *verb_groups*
- *similar_tos*

Additionally, Synsets support the following methods specific to the hypernym relation:

- *root_hypernyms*
- *common_hypernyms*
- *lowest_common_hypernyms*

Note that Synsets do not support the following relations because these are defined by WordNet as Lexical relations:

- *antonyms*
- *derivationally_related_forms*
- *pertainyms*

```
__slots__ = ['_pos', '_offset', '_name', '_frame_ids',
             '_lemmas', '_lemma_names',
             '_definition', '_examples', '_lexname',
             '_pointers', '_lemma_pointers', '_max_depth',
             '_min_depth']
```

```
def __init__(self, wordnet_corpus_reader):
    self.wordnet_corpus_reader = wordnet_corpus_reader
    # All of these attributes get initialized by
    # WordNetCorpusReader._synset_from_pos_and_line()
```

```
self._pos = None
self._offset = None
self._name = None
self._frame_ids = []
self._lemmas = []
self._lemma_names = []
self._definition = None
self._examples = []
self._lexname = None # lexicographer name
self._all_hypernyms = None
```

```
self._pointers = defaultdict(set)
self._lemma_pointers = defaultdict(set)
```

```
def pos(self):
    return self._pos
```

[\[docs\]](#)

```
def offset(self):
    return self._offset
```

[\[docs\]](#)

```

def name(self):
    return self._name

def frame_ids(self):
    return self._frame_ids

def definition(self):
    return self._definition

def examples(self):
    return self._examples

def lexname(self):
    return self._lexname

def _needs_root(self):
    if self._pos == NOUN:
        if self._wordnet_corpus_reader.get_version() == '1.6':
            return True
        else:
            return False
    elif self._pos == VERB:
        return True

def lemma_names(self, lang='eng'):
    '''Return all the lemma_names associated with the synset'''
    if lang == 'eng':
        return self._lemma_names
    else:
        self._wordnet_corpus_reader._load_lang_data(lang)

        i = self._wordnet_corpus_reader.ss2of(self)
        if i in self._wordnet_corpus_reader._lang_data[lang][0]:
            return self._wordnet_corpus_reader._lang_data[lang][0][i]
        else:
            return []

def lemmas(self, lang='eng'):
    '''Return all the Lemma objects associated with the synset'''
    if lang == 'eng':
        return self._lemmas
    else:
        self._wordnet_corpus_reader._load_lang_data(lang)
        lemmark = []
        lemmy = self.lemma_names(lang)
        for lem in lemmy:
            temp = Lemma(
                self._wordnet_corpus_reader,
                self,
                lem,
                self._wordnet_corpus_reader._lexnames.index(
                    self.lexname()
                ),
                0,
                None
            )
            temp._lang = lang
            lemmark.append(temp)
        return lemmark

def root_hyponyms(self):
    '''Get the topmost hyponyms of this synset in WordNet.'''

```

```

result = []
seen = set()
todo = [self]
while todo:
    next_synset = todo.pop()
    if next_synset not in seen:
        seen.add(next_synset)
        next_hyponyms = next_synset.hypernyms() + \
            next_synset.instance_hyponyms()
        if not next_hyponyms:
            result.append(next_synset)
        else:
            todo.extend(next_hyponyms)
return result

# Simpler implementation which makes incorrect assumption that
# hypernym hierarchy is acyclic:
#
#     if not self.hypernyms():
#         return [self]
#     else:
#         return list(set(root for h in self.hypernyms()
#                          for root in h.root_hyponyms()))

def max_depth(self): \[docs\]
    """
    :return: The length of the longest hypernym path from this
    synset to the root.
    """

    if "_max_depth" not in self.__dict__:
        hypernyms = self.hypernyms() + self.instance_hyponyms()
        if not hypernyms:
            self._max_depth = 0
        else:
            self._max_depth = 1 + max(h.max_depth() for h in hypernyms)
    return self._max_depth

def min_depth(self): \[docs\]
    """
    :return: The length of the shortest hypernym path from this
    synset to the root.
    """

    if "_min_depth" not in self.__dict__:
        hypernyms = self.hypernyms() + self.instance_hyponyms()
        if not hypernyms:
            self._min_depth = 0
        else:
            self._min_depth = 1 + min(h.min_depth() for h in hypernyms)
    return self._min_depth

def closure(self, rel, depth=-1): \[docs\]
    """Return the transitive closure of source under the rel
    relationship, breadth-first

    >>> from nltk.corpus import wordnet as wn
    >>> dog = wn.synset('dog.n.01')
    >>> hyp = lambda s:s.hypernyms()
    >>> list(dog.closure(hyp))
    [Synset('canine.n.02'), Synset('domestic_animal.n.01'),
    Synset('carnivore.n.01'), Synset('animal.n.01'),
    Synset('placental.n.01'), Synset('organism.n.01'),
    Synset('mammal.n.01'), Synset('living_thing.n.01'),
    Synset('vertebrate.n.01'), Synset('whole.n.02'),
    Synset('chordate.n.01'), Synset('object.n.01'),
    Synset('physical_entity.n.01'), Synset('entity.n.01')]

```

```

"""
from nltk.util import breadth_first
synset_offsets = []
for synset in breadth_first(self, rel, depth):
    if synset._offset != self._offset:
        if synset._offset not in synset_offsets:
            synset_offsets.append(synset._offset)
        yield synset

def hypernym_paths(self):
    """
    Get the path(s) from this synset to the root, where each path is a
    list of the synset nodes traversed on the way to the root.

    :return: A list of lists, where each list gives the node sequence
             connecting the initial ``Synset`` node and a root node.
    """
    paths = []

    hypernyms = self.hypernyms() + self.instance_hypernyms()
    if len(hypernyms) == 0:
        paths = [[self]]

    for hypernym in hypernyms:
        for ancestor_list in hypernym.hypernym_paths():
            ancestor_list.append(self)
            paths.append(ancestor_list)
    return paths

def common_hypernyms(self, other):
    """
    Find all synsets that are hypernyms of this synset and the
    other synset.

    :type other: Synset
    :param other: other input synset.
    :return: The synsets that are hypernyms of both synsets.
    """
    if not self._all_hypernyms:
        self._all_hypernyms = set(
            self_synset
            for self_synsets in self._iter_hypernym_lists()
            for self_synset in self_synsets
        )
    if not other._all_hypernyms:
        other._all_hypernyms = set(
            other_synset
            for other_synsets in other._iter_hypernym_lists()
            for other_synset in other_synsets
        )
    return list(self._all_hypernyms.intersection(other._all_hypernyms))

def lowest_common_hypernyms(
    self, other, simulate_root=False, use_min_depth=False
):
    """
    Get a list of lowest synset(s) that both synsets have as a hypernym.
    When ``use_min_depth == False`` this means that the synset which appears
    as a hypernym of both ``self`` and ``other`` with the lowest maximum depth
    is returned or if there are multiple such synsets at the same depth
    they are all returned

    However, if ``use_min_depth == True`` then the synset(s) which has/have
    the lowest minimum depth and appear(s) in both paths is/are returned.

    By setting the use_min_depth flag to True, the behavior of NLTK2 can be
    preserved. This was changed in NLTK3 to give more accurate results in a
    small set of cases, generally with synsets concerning people. (eg:

```

[\[docs\]](#)[\[docs\]](#)[\[docs\]](#)

```
'chef.n.01', 'fireman.n.01', etc.)
```

This method is an implementation of Ted Pedersen's "Lowest Common Subsumer" method from the Perl Wordnet module. It can return either "self" or "other" if they are a hypernym of the other.

```
:type other: Synset
:param other: other input synset
:type simulate_root: bool
:param simulate_root: The various verb taxonomies do not
    share a single root which disallows this metric from working for
    synsets that are not connected. This flag (False by default)
    creates a fake root that connects all the taxonomies. Set it
    to True to enable this behavior. For the noun taxonomy,
    there is usually a default root except for WordNet version 1.6.
    If you are using wordnet 1.6, a fake root will need to be added
    for nouns as well.
:type use_min_depth: bool
:param use_min_depth: This setting mimics older (v2) behavior of NLTK
    wordnet. If True, will use the min_depth function to calculate the
    lowest common hypernyms. This is known to give strange results for
    some synset pairs (eg: 'chef.n.01', 'fireman.n.01') but is retained
    for backwards compatibility
:return: The synsets that are the lowest common hypernyms of both
    synsets
"""
```

```
synsets = self.common_hypernyms(other)
if simulate_root:
    fake_synset = Synset(None)
    fake_synset._name = '*ROOT*'
    fake_synset.hypernyms = lambda: []
    fake_synset.instance_hypernyms = lambda: []
    synsets.append(fake_synset)

try:
    if use_min_depth:
        max_depth = max(s.min_depth() for s in synsets)
        unsorted_lch = [
            s for s in synsets if s.min_depth() == max_depth
        ]
    else:
        max_depth = max(s.max_depth() for s in synsets)
        unsorted_lch = [
            s for s in synsets if s.max_depth() == max_depth
        ]
    return sorted(unsorted_lch)
except ValueError:
    return []
```

```
def hypernym_distances(self, distance=0, simulate_root=False): \[docs\]
```

```
"""
Get the path(s) from this synset to the root, counting the distance
of each node from the initial node on the way. A set of
(synset, distance) tuples is returned.

:type distance: int
:param distance: the distance (number of edges) from this hypernym to
    the original hypernym ``Synset`` on which this method was called.
:return: A set of ``(Synset, int)`` tuples where each ``Synset`` is
    a hypernym of the first ``Synset``.
"""
distances = set([(self, distance)])
for hypernym in self._hypernyms() + self._instance_hypernyms():
    distances |= hypernym.hypernym_distances(
        distance+1,
        simulate_root=False
    )
if simulate_root:
    fake_synset = Synset(None)
    fake_synset._name = '*ROOT*'
    distances |= fake_synset.hypernym_distances(
        distance+1,
        simulate_root=False
    )
```

```

fake_synset_distance = max(distances, key=itemgetter(1))[1]
distances.add((fake_synset, fake_synset_distance+1))
return distances

def _shortest_hyponym_paths(self, simulate_root):
    if self._name == '*ROOT*':
        return {self: 0}

    queue = deque([(self, 0)])
    path = {}

    while queue:
        s, depth = queue.popleft()
        if s in path:
            continue
        path[s] = depth

        depth += 1
        queue.extend((hyp, depth) for hyp in s._hyponyms())
        queue.extend((hyp, depth) for hyp in s._instance_hyponyms())

    if simulate_root:
        fake_synset = Synset(None)
        fake_synset._name = '*ROOT*'
        path[fake_synset] = max(path.values()) + 1

    return path

def shortest_path_distance(self, other, simulate_root=False): \[docs\]
    """
    Returns the distance of the shortest path linking the two synsets (if
    one exists). For each synset, all the ancestor nodes and their
    distances are recorded and compared. The ancestor node common to both
    synsets that can be reached with the minimum number of traversals is
    used. If no ancestor nodes are common, None is returned. If a node is
    compared with itself 0 is returned.

    :type other: Synset
    :param other: The Synset to which the shortest path will be found.
    :return: The number of edges in the shortest path connecting the two
             nodes, or None if no path exists.
    """

    if self == other:
        return 0

    dist_dict1 = self._shortest_hyponym_paths(simulate_root)
    dist_dict2 = other._shortest_hyponym_paths(simulate_root)

    # For each ancestor synset common to both subject synsets, find the
    # connecting path length. Return the shortest of these.

    inf = float('inf')
    path_distance = inf
    for synset, d1 in iteritems(dist_dict1):
        d2 = dist_dict2.get(synset, inf)
        path_distance = min(path_distance, d1 + d2)

    return None if math.isinf(path_distance) else path_distance

def tree(self, rel, depth=-1, cut_mark=None): \[docs\]
    """
    >>> from nltk.corpus import wordnet as wn
    >>> dog = wn.synset('dog.n.01')
    >>> hyp = lambda s:s.hypernyms()
    >>> from pprint import pprint
    >>> pprint(dog.tree(hyp))
    [Synset('dog.n.01'),
     [Synset('canine.n.02'),

```

```

[Synset('carnivore.n.01'),
 [Synset('placental.n.01'),
  [Synset('mammal.n.01'),
   [Synset('vertebrate.n.01'),
    [Synset('chordate.n.01'),
     [Synset('animal.n.01'),
      [Synset('organism.n.01'),
       [Synset('living_thing.n.01'),
        [Synset('whole.n.02'),
         [Synset('object.n.01'),
          [Synset('physical_entity.n.01'),
           [Synset('entity.n.01')]]]]]]]]]]],
 [Synset('domestic_animal.n.01'),
 [Synset('animal.n.01'),
  [Synset('organism.n.01'),
   [Synset('living_thing.n.01'),
    [Synset('whole.n.02'),
     [Synset('object.n.01'),
      [Synset('physical_entity.n.01'), [Synset('entity.n.01')]]]]]]]]]
"""

```

```

tree = [self]
if depth != 0:
    tree += [x.tree(rel, depth-1, cut_mark) for x in rel(self)]
elif cut_mark:
    tree += [cut_mark]
return tree

```

interface to similarity methods

```
def path_similarity(self, other, verbose=False, simulate_root=True):
```

[\[docs\]](#)

```

"""
Path Distance Similarity:
Return a score denoting how similar two word senses are, based on the
shortest path that connects the senses in the is-a (hypernym/hypnym)
taxonomy. The score is in the range 0 to 1, except in those cases where
a path cannot be found (will only be true for verbs as there are many
distinct verb taxonomies), in which case None is returned. A score of
1 represents identity i.e. comparing a sense with itself will return 1.

```

```

:type other: Synset
:param other: The ``Synset`` that this ``Synset`` is being compared to.
:type simulate_root: bool
:param simulate_root: The various verb taxonomies do not
    share a single root which disallows this metric from working for
    synsets that are not connected. This flag (True by default)
    creates a fake root that connects all the taxonomies. Set it
    to false to disable this behavior. For the noun taxonomy,
    there is usually a default root except for WordNet version 1.6.
    If you are using wordnet 1.6, a fake root will be added for nouns
    as well.
:return: A score denoting the similarity of the two ``Synset`` objects,
    normally between 0 and 1. None is returned if no connecting path
    could be found. 1 is returned if a ``Synset`` is compared with
    itself.
"""

```

```

distance = self.shortest_path_distance(
    other,
    simulate_root=simulate_root and self._needs_root()
)
if distance is None or distance < 0:
    return None
return 1.0 / (distance + 1)

```

```
def lch_similarity(self, other, verbose=False, simulate_root=True):
```

[\[docs\]](#)

```

"""
Leacock Chodorow Similarity:
Return a score denoting how similar two word senses are, based on the
shortest path that connects the senses (as above) and the maximum depth

```

of the taxonomy in which the senses occur. The relationship is given as $-\log(p/2d)$ where p is the shortest path length and d is the taxonomy depth.

```
:type other: Synset
:param other: The ``Synset`` that this ``Synset`` is being compared to.
:type simulate_root: bool
:param simulate_root: The various verb taxonomies do not
    share a single root which disallows this metric from working for
    synsets that are not connected. This flag (True by default)
    creates a fake root that connects all the taxonomies. Set it
    to false to disable this behavior. For the noun taxonomy,
    there is usually a default root except for WordNet version 1.6.
    If you are using wordnet 1.6, a fake root will be added for nouns
    as well.
:return: A score denoting the similarity of the two ``Synset`` objects,
    normally greater than 0. None is returned if no connecting path
    could be found. If a ``Synset`` is compared with itself, the
    maximum score is returned, which varies depending on the taxonomy
    depth.
```

```
"""
```

```
if self._pos != other._pos:
    raise WordNetError(
        'Computing the lch similarity requires '
        '%s and %s to have the same part of speech.' %
        (self, other)
    )
```

```
need_root = self._needs_root()
```

```
if self._pos not in self._wordnet_corpus_reader._max_depth:
    self._wordnet_corpus_reader._compute_max_depth(
        self._pos, need_root
    )
```

```
depth = self._wordnet_corpus_reader._max_depth[self._pos]
```

```
distance = self.shortest_path_distance(
    other,
    simulate_root=simulate_root and need_root
)
```

```
if distance is None or distance < 0 or depth == 0:
    return None
return -math.log((distance + 1) / (2.0 * depth))
```

```
def wup_similarity(self, other, verbose=False, simulate_root=True):
    """
```

[\[docs\]](#)

Wu-Palmer Similarity:
Return a score denoting how similar two word senses are, based on the depth of the two senses in the taxonomy and that of their Least Common Subsumer (most specific ancestor node). Previously, the scores computed by this implementation did *not* always agree with those given by Pedersen's Perl implementation of WordNet Similarity. However, with the addition of the `simulate_root` flag (see below), the score for verbs now almost always agree but not always for nouns.

The LCS does not necessarily feature in the shortest path connecting the two senses, as it is by definition the common ancestor deepest in the taxonomy, not closest to the two senses. Typically, however, it will so feature. Where multiple candidates for the LCS exist, that whose shortest path to the root node is the longest will be selected. Where the LCS has multiple paths to the root, the longer path is used for the purposes of the calculation.

```
:type other: Synset
:param other: The ``Synset`` that this ``Synset`` is being compared to.
:type simulate_root: bool
:param simulate_root: The various verb taxonomies do not
```

share a single root which disallows this metric from working for synsets that are not connected. This flag (True by default) creates a fake root that connects all the taxonomies. Set it to false to disable this behavior. For the noun taxonomy, there is usually a default root except for WordNet version 1.6. If you are using wordnet 1.6, a fake root will be added for nouns as well.

:return: A float score denoting the similarity of the two ``Synset`` objects, normally greater than zero. If no connecting path between the two senses can be found, None is returned.

"""

```

need_root = self._needs_root()
# Note that to preserve behavior from NLTK2 we set use_min_depth=True
# It is possible that more accurate results could be obtained by
# removing this setting and it should be tested later on
subsumers = self.lowest_common_hyponyms(
    other,
    simulate_root=simulate_root and need_root, use_min_depth=True
)

```

```

# If no LCS was found return None
if len(subsumers) == 0:
    return None

```

```

subsumer = subsumers[0]

```

```

# Get the longest path from the LCS to the root,
# including a correction:
# - add one because the calculations include both the start and end
# nodes
depth = subsumer.max_depth() + 1

```

```

# Note: No need for an additional add-one correction for non-nouns
# to account for an imaginary root node because that is now
# automatically handled by simulate_root
# if subsumer._pos != NOUN:
#     depth += 1

```

```

# Get the shortest path from the LCS to each of the synsets it is
# subsuming. Add this to the LCS path length to get the path
# length from each synset to the root.

```

```

len1 = self.shortest_path_distance(
    subsumer,
    simulate_root=simulate_root and need_root
)
len2 = other.shortest_path_distance(
    subsumer,
    simulate_root=simulate_root and need_root
)
if len1 is None or len2 is None:
    return None
len1 += depth
len2 += depth
return (2.0 * depth) / (len1 + len2)

```

```
def res_similarity(self, other, ic, verbose=False):
```

[\[docs\]](#)

"""

Resnik Similarity:
Return a score denoting how similar two word senses are, based on the Information Content (IC) of the Least Common Subsumer (most specific ancestor node).

```

:type other: Synset
:param other: The ``Synset`` that this ``Synset`` is being compared to.
:type ic: dict
:param ic: an information content object (as returned by
    ``nltk.corpus.wordnet_ic.ic()``).
:return: A float score denoting the similarity of the two ``Synset``

```

objects. Synsets whose LCS is the root node of the taxonomy will have a score of 0 (e.g. `N['dog'][0]` and `N['table'][0]`).

"""

```
ic1, ic2, lcs_ic = _lcs_ic(self, other, ic)
return lcs_ic
```

```
def jcn_similarity(self, other, ic, verbose=False): \[docs\]
```

"""

Jiang-Conrath Similarity:

*Return a score denoting how similar two word senses are, based on the Information Content (IC) of the Least Common Subsumer (most specific ancestor node) and that of the two input Synsets. The relationship is given by the equation $1 / (IC(s1) + IC(s2) - 2 * IC(Lcs))$.*

:type other: Synset

:param other: The ``Synset`` that this ``Synset`` is being compared to.

:type ic: dict

:param ic: an information content object (as returned by

```nlk.corpus.wordnet_ic.ic()```).

:return: A float score denoting the similarity of the two ``Synset`` objects.

"""

```
if self == other:
```

```
    return _INF
```

```
ic1, ic2, lcs_ic = _lcs_ic(self, other, ic)
```

If either of the input synsets are the root synset, or have a

frequency of 0 (sparse data problem), return 0.

```
if ic1 == 0 or ic2 == 0:
```

```
    return 0
```

```
ic_difference = ic1 + ic2 - 2 * lcs_ic
```

```
if ic_difference == 0:
```

```
    return _INF
```

```
return 1 / ic_difference
```

```
def lin_similarity(self, other, ic, verbose=False): \[docs\]
```

"""

Lin Similarity:

*Return a score denoting how similar two word senses are, based on the Information Content (IC) of the Least Common Subsumer (most specific ancestor node) and that of the two input Synsets. The relationship is given by the equation $2 * IC(Lcs) / (IC(s1) + IC(s2))$.*

:type other: Synset

:param other: The ``Synset`` that this ``Synset`` is being compared to.

:type ic: dict

:param ic: an information content object (as returned by

```nlk.corpus.wordnet_ic.ic()```).

:return: A float score denoting the similarity of the two ``Synset`` objects, in the range 0 to 1.

"""

```
ic1, ic2, lcs_ic = _lcs_ic(self, other, ic)
```

```
return (2.0 * lcs_ic) / (ic1 + ic2)
```

```
def _iter_hyponym_lists(self):
```

"""

:return: An iterator over ``Synset`` objects that are either proper hyponyms or instance of hyponyms of the synset.

"""

```
todo = [self]
```

```
seen = set()
```

```

while todo:
    for synset in todo:
        seen.add(synset)
    yield todo
    todo = [hypernym
             for synset in todo
             for hypernym in (
                 synset.hypernyms() + synset.instance_hypernyms()
             )
             if hypernym not in seen]

def __repr__(self):
    return "%s('%s')" % (type(self).__name__, self._name)

def _related(self, relation_symbol, sort=True):
    get_synset = self._wordnet_corpus_reader.synset_from_pos_and_offset
    pointer_tuples = self._pointers[relation_symbol]
    r = [get_synset(pos, offset) for pos, offset in pointer_tuples]
    if sort:
        r.sort()
    return r

#####
# WordNet Corpus Reader
#####

class WordNetCorpusReader(CorpusReader): \[docs\]
    """
    A corpus reader used to access wordnet or its variants.
    """

    _ENCODING = 'utf8'

    # { Part-of-speech constants
    ADJ, ADJ_SAT, ADV, NOUN, VERB = 'a', 's', 'r', 'n', 'v'
    # }

    # { Filename constants
    _FILEMAP = {ADJ: 'adj', ADV: 'adv', NOUN: 'noun', VERB: 'verb'}
    # }

    # { Part of speech constants
    _pos_numbers = {NOUN: 1, VERB: 2, ADJ: 3, ADV: 4, ADJ_SAT: 5}
    _pos_names = dict(tup[::-1] for tup in _pos_numbers.items())
    # }

    #: A list of file identifiers for all the fileids used by this
    #: corpus reader.
    _FILES = ('cntlist.rev', 'lexnames', 'index.sense',
              'index.adj', 'index.adv', 'index.noun', 'index.verb',
              'data.adj', 'data.adv', 'data.noun', 'data.verb',
              'adj.exc', 'adv.exc', 'noun.exc', 'verb.exc', )

    def __init__(self, root, omw_reader):
        """
        Construct a new wordnet corpus reader, with the given root
        directory.
        """
        super(WordNetCorpusReader, self).__init__(root, self._FILES,
                                                  encoding=self._ENCODING)

        # A index that provides the file offset
        # Map from Lemma -> pos -> synset_index -> offset
        self._lemma_pos_offset_map = defaultdict(dict)

        # A cache so we don't have to reconstuct synsets
        # Map from pos -> offset -> synset
        self._synset_offset_cache = defaultdict(dict)

```

```

# A Lookup for the maximum depth of each part of speech. Useful for
# the Lch similarity metric.
self._max_depth = defaultdict(dict)

# Corpus reader containing omw data.
self._omw_reader = omw_reader

# A cache to store the wordnet data of multiple languages
self._lang_data = defaultdict(list)

self._data_file_map = {}
self._exception_map = {}
self._lexnames = []
self._key_count_file = None
self._key_synset_file = None

# Load the Lexnames
for i, line in enumerate(self.open('lexnames')):
    index, lexname, _ = line.split()
    assert int(index) == i
    self._lexnames.append(lexname)

# Load the indices for Lemmas and synset offsets
self._load_lemma_pos_offset_map()

# Load the exception file data into memory
self._load_exception_map()

# Open Multilingual WordNet functions, contributed by
# Nasruddin A'aidil Shari, Sim Wei Ying Geraldine, and Soe Lynn

def of2ss(self, of):
    ''' take an id and return the synsets '''
    return self.synset_from_pos_and_offset(of[-1], int(of[:8]))

def ss2of(self, ss):
    ''' return the ID of the synset '''
    return "{:08d}-{}".format(ss.offset(), ss.pos())

def _load_lang_data(self, lang):
    ''' Load the wordnet data of the requested language from the file to
    the cache, _Lang_data '''

    if lang in self._lang_data.keys():
        return

    if lang not in self.langs():
        raise WordNetError("Language is not supported.")

    f = self._omw_reader.open('{0:}/wn-data-{0:}.tab'.format(lang))
    self.custom_lemmas(f, lang)
    f.close()

def langs(self):
    ''' return a List of Languages supported by Multilingual Wordnet '''
    import os
    langs = ['eng']
    fileids = self._omw_reader.fileids()
    for fileid in fileids:
        file_name, file_extension = os.path.splitext(fileid)
        if file_extension == '.tab':
            langs.append(file_name.split('-')[-1])

    return langs

def _load_lemma_pos_offset_map(self):
    for suffix in self._FILEMAP.values():

```

[\[docs\]](#)[\[docs\]](#)[\[docs\]](#)

```

# parse each line of the file (ignoring comment lines)
for i, line in enumerate(self.open('index.%s' % suffix)):
    if line.startswith(' '):
        continue

    _iter = iter(line.split())

    def _next_token(): return next(_iter)

    try:

        # get the lemma and part-of-speech
        lemma = _next_token()
        pos = _next_token()

        # get the number of synsets for this lemma
        n_synsets = int(_next_token())
        assert n_synsets > 0

        # get and ignore the pointer symbols for all synsets of
        # this lemma
        n_pointers = int(_next_token())
        [_next_token() for _ in range(n_pointers)]

        # same as number of synsets
        n_senses = int(_next_token())
        assert n_synsets == n_senses

        # get and ignore number of senses ranked according to
        # frequency
        _next_token()

        # get synset offsets
        synset_offsets = [
            int(_next_token()) for _ in range(n_synsets)
        ]

        # raise more informative error with file name and line number
        except (AssertionError, ValueError) as e:
            tup = ('index.%s' % suffix), (i + 1), e
            raise WordNetError('file %s, line %i: %s' % tup)

        # map lemmas and parts of speech to synsets
        self._lemma_pos_offset_map[lemma][pos] = synset_offsets
        if pos == ADJ:
            self._lemma_pos_offset_map[lemma][ADJ_SAT] = synset_offsets

def _load_exception_map(self):
    # Load the exception file data into memory
    for pos, suffix in self._FILEMAP.items():
        self._exception_map[pos] = {}
        for line in self.open('%s.exc' % suffix):
            terms = line.split()
            self._exception_map[pos][terms[0]] = terms[1:]
    self._exception_map[ADJ_SAT] = self._exception_map[ADJ]

def _compute_max_depth(self, pos, simulate_root):
    """
    Compute the max depth for the given part of speech. This is
    used by the lch similarity metric.
    """
    depth = 0
    for ii in self.all_synsets(pos):
        try:
            depth = max(depth, ii.max_depth())
        except RuntimeError:
            print(ii)
    if simulate_root:
        depth += 1
    self._max_depth[pos] = depth

```

```
def get_version(self):
    fh = self._data_file(ADJ)
    for line in fh:
        match = re.search(r'WordNet (\d+\.\d+) Copyright', line)
        if match is not None:
            version = match.group(1)
            fh.seek(0)
            return version
```

[\[docs\]](#)

```
#####
# Loading Lemmas
#####
```

```
def lemma(self, name, lang='eng'):
    '''Return Lemma object that matches the name'''
    # cannot simply split on first '.',
    # e.g.: '.45_caliber.a.01..45_caliber'
    separator = SENSENUM_RE.search(name).start()
    synset_name, lemma_name = name[:separator+3], name[separator+4:]
    synset = self.synset(synset_name)
    for lemma in synset.lemmas(lang):
        if lemma._name == lemma_name:
            return lemma
    raise WordNetError('no lemma %r in %r' % (lemma_name, synset_name))
```

[\[docs\]](#)

```
def lemma_from_key(self, key):
    # Keys are case sensitive and always lower-case
    key = key.lower()

    lemma_name, lex_sense = key.split('%')
    pos_number, lexname_index, lex_id, _, _ = lex_sense.split(':')
    pos = self._pos_names[int(pos_number)]

    # open the key -> synset file if necessary
    if self._key_synset_file is None:
        self._key_synset_file = self.open('index.sense')

    # Find the synset for the Lemma.
    synset_line = _binary_search_file(self._key_synset_file, key)
    if not synset_line:
        raise WordNetError("No synset found for key %r" % key)
    offset = int(synset_line.split()[1])
    synset = self.synset_from_pos_and_offset(pos, offset)

    # return the corresponding Lemma
    for lemma in synset._lemmas:
        if lemma._key == key:
            return lemma
    raise WordNetError("No lemma found for for key %r" % key)
```

[\[docs\]](#)

```
#####
# Loading Synsets
#####
```

```
def synset(self, name):
    # split name into lemma, part of speech and synset number
    lemma, pos, synset_index_str = name.lower().rsplit('.', 2)
    synset_index = int(synset_index_str) - 1

    # get the offset for this synset
    try:
        offset = self._lemma_pos_offset_map[lemma][pos][synset_index]
    except KeyError:
        message = 'no lemma %r with part of speech %r'
        raise WordNetError(message % (lemma, pos))
    except IndexError:
        n_senses = len(self._lemma_pos_offset_map[lemma][pos])
        message = "lemma %r with part of speech %r has only %i %s"
        if n_senses == 1:
```

[\[docs\]](#)

```

        tup = lemma, pos, n_senses, "sense"
    else:
        tup = lemma, pos, n_senses, "senses"
    raise WordNetError(message % tup)

# Load synset information from the appropriate file
synset = self.synset_from_pos_and_offset(pos, offset)

# some basic sanity checks on loaded attributes
if pos == 's' and synset._pos == 'a':
    message = ('adjective satellite requested but only plain '
               'adjective found for lemma %r')
    raise WordNetError(message % lemma)
assert synset._pos == pos or (pos == 'a' and synset._pos == 's')

# Return the synset object.
return synset

def _data_file(self, pos):
    """
    Return an open file pointer for the data file for the given
    part of speech.
    """
    if pos == ADJ_SAT:
        pos = ADJ
    if self._data_file_map.get(pos) is None:
        fileid = 'data.%s' % self._FILEMAP[pos]
        self._data_file_map[pos] = self.open(fileid)
    return self._data_file_map[pos]

def synset_from_pos_and_offset(self, pos, offset): \[docs\]
    # Check to see if the synset is in the cache
    if offset in self._synset_offset_cache[pos]:
        return self._synset_offset_cache[pos][offset]

    data_file = self._data_file(pos)
    data_file.seek(offset)
    data_file_line = data_file.readline()
    synset = self._synset_from_pos_and_line(pos, data_file_line)
    assert synset._offset == offset
    self._synset_offset_cache[pos][offset] = synset
    return synset

@deprecated('Use public method synset_from_pos_and_offset() instead')
def _synset_from_pos_and_offset(self, *args, **kwargs):
    """
    Hack to help people like the readers of
    http://stackoverflow.com/a/27145655/1709587
    who were using this function before it was officially a public method
    """
    return self.synset_from_pos_and_offset(*args, **kwargs)

def _synset_from_pos_and_line(self, pos, data_file_line):
    # Construct a new (empty) synset.
    synset = Synset(self)

    # parse the entry for this synset
    try:
        # parse out the definitions and examples from the gloss
        columns_str, gloss = data_file_line.split('|')
        gloss = gloss.strip()
        definitions = []
        for gloss_part in gloss.split(';'):
            gloss_part = gloss_part.strip()
            if gloss_part.startswith(''):
                synset._examples.append(gloss_part.strip(''))
            else:
                definitions.append(gloss_part)

```

```

synset._definition = '; '.join(definitions)

# split the other info into fields
_iter = iter(columns_str.split())

def _next_token(): return next(_iter)

# get the offset
synset._offset = int(_next_token())

# determine the lexicographer file name
lexname_index = int(_next_token())
synset._lexname = self._lexnames[lexname_index]

# get the part of speech
synset._pos = _next_token()

# create Lemma objects for each Lemma
n_lemmas = int(_next_token(), 16)
for _ in range(n_lemmas):
    # get the lemma name
    lemma_name = _next_token()
    # get the lex_id (used for sense_keys)
    lex_id = int(_next_token(), 16)
    # If the lemma has a syntactic marker, extract it.
    m = re.match(r'(.*)\((.*)\)?$', lemma_name)
    lemma_name, syn_mark = m.groups()
    # create the lemma object
    lemma = Lemma(self, synset, lemma_name, lexname_index,
                  lex_id, syn_mark)
    synset._lemmas.append(lemma)
    synset._lemma_names.append(lemma._name)

# collect the pointer tuples
n_pointers = int(_next_token())
for _ in range(n_pointers):
    symbol = _next_token()
    offset = int(_next_token())
    pos = _next_token()
    lemma_ids_str = _next_token()
    if lemma_ids_str == '0000':
        synset._pointers[symbol].add((pos, offset))
    else:
        source_index = int(lemma_ids_str[:2], 16) - 1
        target_index = int(lemma_ids_str[2:], 16) - 1
        source_lemma_name = synset._lemmas[source_index]._name
        lemma_pointers = synset._lemma_pointers
        tups = lemma_pointers[source_lemma_name, symbol]
        tups.add((pos, offset, target_index))

# read the verb frames
try:
    frame_count = int(_next_token())
except StopIteration:
    pass
else:
    for _ in range(frame_count):
        # read the plus sign
        plus = _next_token()
        assert plus == '+'
        # read the frame and Lemma number
        frame_number = int(_next_token())
        frame_string_fmt = VERB_FRAME_STRINGS[frame_number]
        lemma_number = int(_next_token(), 16)
        # lemma number of 00 means all words in the synset
        if lemma_number == 0:
            synset._frame_ids.append(frame_number)
            for lemma in synset._lemmas:
                lemma._frame_ids.append(frame_number)
                lemma._frame_strings.append(
                    frame_string_fmt % lemma._name

```

```

    )
    # only a specific word in the synset
    else:
        lemma = synset._lemmas[lemma_number - 1]
        lemma._frame_ids.append(frame_number)
        lemma._frame_strings.append(
            frame_string_fmt % lemma._name
        )

# raise a more informative error with line text
except ValueError as e:
    raise WordNetError('line %r: %s' % (data_file_line, e))

# set sense keys for Lemma objects - note that this has to be
# done afterwards so that the relations are available
for lemma in synset._lemmas:
    if synset._pos == ADJ_SAT:
        head_lemma = synset.similar_tos()[0]._lemmas[0]
        head_name = head_lemma._name
        head_id = '%02d' % head_lemma._lex_id
    else:
        head_name = head_id = ''
    tup = (lemma._name, WordNetCorpusReader._pos_numbers[synset._pos],
           lemma._lexname_index, lemma._lex_id, head_name, head_id)
    lemma._key = ('%s%%d:%02d:%02d:%s:%s' % tup).lower()

# the canonical name is based on the first Lemma
lemma_name = synset._lemmas[0]._name.lower()
offsets = self._lemma_pos_offset_map[lemma_name][synset._pos]
sense_index = offsets.index(synset._offset)
tup = lemma_name, synset._pos, sense_index + 1
synset._name = '%s.%s.%02i' % tup

return synset

#####
# Retrieve synsets and lemmas.
#####

def synsets(self, lemma, pos=None, lang='eng', check_exceptions=True): \[docs\]
    """Load all synsets with a given Lemma and part of speech tag.
    If no pos is specified, all synsets for all parts of speech
    will be loaded.
    If lang is specified, all the synsets associated with the Lemma name
    of that language will be returned.
    """
    lemma = lemma.lower()

    if lang == 'eng':
        get_synset = self.synset_from_pos_and_offset
        index = self._lemma_pos_offset_map
        if pos is None:
            pos = POS_LIST
        return [get_synset(p, offset)
                for p in pos
                for form in self._morphology(lemma, p, check_exceptions)
                for offset in index[form].get(p, [])]

    else:
        self._load_lang_data(lang)
        synset_list = []
        for l in self._lang_data[lang][1][lemma]:
            if pos is not None and l[-1] != pos:
                continue
            synset_list.append(self.of2ss(l))
        return synset_list

def lemmas(self, lemma, pos=None, lang='eng'): \[docs\]
    """Return all Lemma objects with a name matching the specified Lemma
    name and part of speech tag. Matches any part of speech tag if none is

```

specified."""

```
lemma = lemma.lower()
if lang == 'eng':
    return [lemma_obj
            for synset in self.synsets(lemma, pos)
            for lemma_obj in synset.lemmas()
            if lemma_obj.name().lower() == lemma]
```

```
else:
    self._load_lang_data(lang)
    lemmas = []
    syn = self.synsets(lemma, lang=lang)
    for s in syn:
        if pos is not None and s.pos() != pos:
            continue
        for lemma_obj in s.lemmas(lang=lang):
            if lemma_obj.name().lower() == lemma:
                lemmas.append(lemma_obj)
    return lemmas
```

```
def all_lemma_names(self, pos=None, lang='eng'): \[docs\]
```

"""Return all lemma names for all synsets for the given part of speech tag and language or languages. If pos is not specified, all synsets for all parts of speech will be used."""

```
if lang == 'eng':
    if pos is None:
        return iter(self._lemma_pos_offset_map)
    else:
        return (
            lemma for lemma in self._lemma_pos_offset_map
            if pos in self._lemma_pos_offset_map[lemma]
        )
else:
    self._load_lang_data(lang)
    lemma = []
    for i in self._lang_data[lang][0]:
        if pos is not None and i[-1] != pos:
            continue
        lemma.extend(self._lang_data[lang][0][i])

    lemma = list(set(lemma))
    return lemma
```

```
def all_synsets(self, pos=None): \[docs\]
```

"""Iterate over all synsets with a given part of speech tag. If no pos is specified, all synsets for all parts of speech will be loaded."""

```
if pos is None:
    pos_tags = self._FILEMAP.keys()
else:
    pos_tags = [pos]
```

```
cache = self._synset_offset_cache
from_pos_and_line = self._synset_from_pos_and_line
```

```
# generate all synsets for each part of speech
for pos_tag in pos_tags:
    # Open the file for reading. Note that we can not re-use
    # the file pointers from self._data_file_map here, because
    # we're defining an iterator, and those file pointers might
    # be moved while we're not looking.
    if pos_tag == ADJ_SAT:
        pos_tag = ADJ
    fileid = 'data.%s' % self._FILEMAP[pos_tag]
    data_file = self.open(fileid)
```

```

try:
    # generate synsets for each line in the POS file
    offset = data_file.tell()
    line = data_file.readline()
    while line:
        if not line[0].isspace():
            if offset in cache[pos_tag]:
                # See if the synset is cached
                synset = cache[pos_tag][offset]
            else:
                # Otherwise, parse the line
                synset = from_pos_and_line(pos_tag, line)
                cache[pos_tag][offset] = synset

            # adjective satellites are in the same file as
            # adjectives so only yield the synset if it's actually
            # a satellite
            if synset._pos == ADJ_SAT:
                yield synset

            # for all other POS tags, yield all synsets (this means
            # that adjectives also include adjective satellites)
            else:
                yield synset
        offset = data_file.tell()
        line = data_file.readline()

# close the extra file handle we opened
except:
    data_file.close()
    raise
else:
    data_file.close()

```

```

def words(self, lang='eng'):
    """Return lemmas of the given language as list of words"""
    return self.all_lemma_names(lang=lang)

```

[\[docs\]](#)

```

def license(self, lang='eng'):
    """Return the contents of LICENSE (for omw)
    use lang=lang to get the license for an individual language"""
    if lang == 'eng':
        return self.open("LICENSE").read()
    elif lang in self.langs():
        return self._omw_reader.open("{} / LICENSE".format(lang)).read()
    elif lang == 'omw':
        # under the assumption you don't mean Omwunra-Toqura
        return self._omw_reader.open("LICENSE").read()
    elif lang in self._lang_data:
        raise WordNetError(
            "Cannot determine license for user-provided tab file"
        )
    else:
        raise WordNetError("Language is not supported.")

```

[\[docs\]](#)

```

def readme(self, lang='omw'):
    """Return the contents of README (for omw)
    use lang=lang to get the readme for an individual language"""
    if lang == 'eng':
        return self.open("README").read()
    elif lang in self.langs():
        return self._omw_reader.open("{} / README".format(lang)).read()
    elif lang == 'omw':
        # under the assumption you don't mean Omwunra-Toqura
        return self._omw_reader.open("README").read()
    elif lang in self._lang_data:
        raise WordNetError("No README for user-provided tab file")

```

[\[docs\]](#)

```

else:
    raise WordNetError("Language is not supported.")

def citation(self, lang='omw'): [docs]
    """Return the contents of citation.bib file (for omw)
    use lang=lang to get the citation for an individual language"""
    if lang == 'eng':
        return self.open("citation.bib").read()
    elif lang in self.langs():
        return self._omw_reader.open("{}citation.bib".format(lang)).read()
    elif lang == 'omw':
        # under the assumption you don't mean Omwunra-Toqura
        return self._omw_reader.open("citation.bib").read()
    elif lang in self._lang_data:
        raise WordNetError("citation not known for user-provided tab file")
    else:
        raise WordNetError("Language is not supported.")

#####
# Misc
#####
def lemma_count(self, lemma): [docs]
    """Return the frequency count for this Lemma"""
    # Currently, count is only work for English
    if lemma._lang != 'eng':
        return 0
    # open the count file if we haven't already
    if self._key_count_file is None:
        self._key_count_file = self.open('cntlist.rev')
    # find the key in the counts file and return the count
    line = _binary_search_file(self._key_count_file, lemma._key)
    if line:
        return int(line.rsplit(' ', 1)[-1])
    else:
        return 0

def path_similarity( [docs]
    self, synset1, synset2, verbose=False, simulate_root=True
):
    return synset1.path_similarity(synset2, verbose, simulate_root)

path_similarity.__doc__ = Synset.path_similarity.__doc__

def lch_similarity( [docs]
    self, synset1, synset2, verbose=False, simulate_root=True
):
    return synset1.lch_similarity(synset2, verbose, simulate_root)

lch_similarity.__doc__ = Synset.lch_similarity.__doc__

def wup_similarity( [docs]
    self, synset1, synset2, verbose=False, simulate_root=True
):
    return synset1.wup_similarity(synset2, verbose, simulate_root)

wup_similarity.__doc__ = Synset.wup_similarity.__doc__

def res_similarity(self, synset1, synset2, ic, verbose=False): [docs]
    return synset1.res_similarity(synset2, ic, verbose)

res_similarity.__doc__ = Synset.res_similarity.__doc__

def jcn_similarity(self, synset1, synset2, ic, verbose=False): [docs]
    return synset1.jcn_similarity(synset2, ic, verbose)

jcn_similarity.__doc__ = Synset.jcn_similarity.__doc__

```

```
def lin_similarity(self, synset1, synset2, ic, verbose=False): \[docs\]
    return synset1.lin_similarity(synset2, ic, verbose)
```

```
lin_similarity.__doc__ = Synset.lin_similarity.__doc__
```

```
#####
```

```
# Morphy
```

```
#####
```

```
# Morphy, adapted from Oliver Steele's pywordnet
```

```
def morphy(self, form, pos=None, check_exceptions=True): \[docs\]
```

```
    """
```

```
    Find a possible base form for the given form, with the given
    part of speech, by checking WordNet's List of exceptional
    forms, and by recursively stripping affixes for this part of
    speech until a form in WordNet is found.
```

```
>>> from nltk.corpus import wordnet as wn
```

```
>>> print(wn.morphy('dogs'))
```

```
dog
```

```
>>> print(wn.morphy('churches'))
```

```
church
```

```
>>> print(wn.morphy('aardwolves'))
```

```
aardwolf
```

```
>>> print(wn.morphy('abaci'))
```

```
abacus
```

```
>>> wn.morphy('hardrock', wn.ADV)
```

```
>>> print(wn.morphy('book', wn.NOUN))
```

```
book
```

```
>>> wn.morphy('book', wn.ADJ)
```

```
"""
```

```
if pos is None:
```

```
    morphy = self._morphy
```

```
    analyses = chain(a for p in POS_LIST for a in morphy(form, p))
```

```
else:
```

```
    analyses = self._morphy(form, pos, check_exceptions)
```

```
# get the first one we find
```

```
first = list(islice(analyses, 1))
```

```
if len(first) == 1:
```

```
    return first[0]
```

```
else:
```

```
    return None
```

```
MORPHOLOGICAL_SUBSTITUTIONS = {
```

```
    NOUN: [('s', ''), ('ses', 's'), ('ves', 'f'), ('xes', 'x'),
           ('zes', 'z'), ('ches', 'ch'), ('shes', 'sh'),
           ('men', 'man'), ('ies', 'y')],
```

```
    VERB: [('s', ''), ('ies', 'y'), ('es', 'e'), ('es', ''),
           ('ed', 'e'), ('ed', ''), ('ing', 'e'), ('ing', '')],
```

```
    ADJ: [('er', ''), ('est', ''), ('er', 'e'), ('est', 'e')],
```

```
    ADV: []}
```

```
MORPHOLOGICAL_SUBSTITUTIONS[ADJ_SAT] = MORPHOLOGICAL_SUBSTITUTIONS[ADJ]
```

```
def _morphy(self, form, pos, check_exceptions=True):
```

```
    # from jordanbg:
```

```
    # Given an original string x
```

```
    # 1. Apply rules once to the input to get y1, y2, y3, etc.
```

```
    # 2. Return all that are in the database
```

```
    # 3. If there are no matches, keep applying rules until you either
```

```
    # find a match or you can't go any further
```

```
exceptions = self._exception_map[pos]
```

```
substitutions = self.MORPHOLOGICAL_SUBSTITUTIONS[pos]
```

```
def apply_rules(forms):
```

```
    return [form[:-len(old)] + new
```

```
            for form in forms
```

```
            for old, new in substitutions
```

```

        if form.endswith(old)]

def filter_forms(forms):
    result = []
    seen = set()
    for form in forms:
        if form in self._lemma_pos_offset_map:
            if pos in self._lemma_pos_offset_map[form]:
                if form not in seen:
                    result.append(form)
                    seen.add(form)
    return result

# 0. Check the exception lists
if check_exceptions:
    if form in exceptions:
        return filter_forms([form] + exceptions[form])

# 1. Apply rules once to the input to get y1, y2, y3, etc.
forms = apply_rules([form])

# 2. Return all that are in the database (and check the original too)
results = filter_forms([form] + forms)
if results:
    return results

# 3. If there are no matches, keep applying rules until we find a match
while forms:
    forms = apply_rules(forms)
    results = filter_forms(forms)
    if results:
        return results

# Return an empty List if we can't find anything
return []

#####
# Create information content from corpus
#####
def ic(self, corpus, weight_senses_equally=False, smoothing=1.0): \[docs\]
    """
    Creates an information content lookup dictionary from a corpus.

    :type corpus: CorpusReader
    :param corpus: The corpus from which we create an information
    content dictionary.
    :type weight_senses_equally: bool
    :param weight_senses_equally: If this is True, gives all
    possible senses equal weight rather than dividing by the
    number of possible senses. (If a word has 3 synses, each
    sense gets 0.3333 per appearance when this is False, 1.0 when
    it is true.)
    :param smoothing: How much do we smooth synset counts (default is 1.0)
    :type smoothing: float
    :return: An information content dictionary
    """
    counts = FreqDist()
    for ww in corpus.words():
        counts[ww] += 1

    ic = {}
    for pp in POS_LIST:
        ic[pp] = defaultdict(float)

    # Initialize the counts with the smoothing value
    if smoothing > 0.0:
        for ss in self.all_synsets():
            pos = ss._pos
            if pos == ADJ_SAT:
                pos = ADJ
            ic[pos][ss._offset] = smoothing

```

```

for ww in counts:
    possible_synsets = self.synsets(ww)
    if len(possible_synsets) == 0:
        continue

    # Distribute weight among possible synsets
    weight = float(counts[ww])
    if not weight_senses_equally:
        weight /= float(len(possible_synsets))

    for ss in possible_synsets:
        pos = ss._pos
        if pos == ADJ_SAT:
            pos = ADJ
        for level in ss._iter_hypernym_lists():
            for hh in level:
                ic[pos][hh._offset] += weight
            # Add the weight to the root
            ic[pos][0] += weight
    return ic

def custom_lemmas(self, tab_file, lang):
    """
    Reads a custom tab file containing mappings of lemmas in the given
    language to Princeton WordNet 3.0 synset offsets, allowing NLTK's
    WordNet functions to then be used with that language.

    See the "Tab files" section at http://compling.hss.ntu.edu.sg/omw/ for
    documentation on the Multilingual WordNet tab file format.

    :param tab_file: Tab file as a file or file-like object
    :type lang str
    :param lang ISO 639-3 code of the language of the tab file
    """
    if len(lang) != 3:
        raise ValueError('lang should be a (3 character) ISO 639-3 code')
    self._lang_data[lang] = [defaultdict(list), defaultdict(list)]
    for l in tab_file.readlines():
        if isinstance(l, bytes):
            # Support byte-stream files (e.g. as returned by Python 2's
            # open() function) as well as text-stream ones
            l = l.decode('utf-8')
        l = l.replace('\n', '')
        l = l.replace(' ', '_')
        if l[0] != '#':
            word = l.split('\t')
            self._lang_data[lang][0][word[0]].append(word[2])
            self._lang_data[lang][1][word[2].lower()].append(word[0])

#####
# WordNet Information Content Corpus Reader
#####

class WordNetICCorpusReader(CorpusReader):
    """
    A corpus reader for the WordNet information content corpus.
    """

    def __init__(self, root, fileids):
        CorpusReader.__init__(self, root, fileids, encoding='utf8')

    # this load function would be more efficient if the data was pickled
    # Note that we can't use NLTK's frequency distributions because
    # synsets are overlapping (each instance of a synset also counts
    # as an instance of its hypernyms)
    def ic(self, icfile):

```

[\[docs\]](#)[\[docs\]](#)[\[docs\]](#)

Load an information content file from the wordnet_ic corpus and return a dictionary. This dictionary has just two keys, NOUN and VERB, whose values are dictionaries that map from synsets to information content values.

```
:type icfile: str
:param icfile: The name of the wordnet_ic file (e.g. "ic-brown.dat")
:return: An information content dictionary
"""
ic = {}
ic[NOUN] = defaultdict(float)
ic[VERB] = defaultdict(float)
for num, line in enumerate(self.open(icfile)):
    if num == 0: # skip the header
        continue
    fields = line.split()
    offset = int(fields[0][: -1])
    value = float(fields[1])
    pos = _get_pos(fields[0])
    if len(fields) == 3 and fields[2] == "ROOT":
        # Store root count.
        ic[pos][0] += value
    if value != 0:
        ic[pos][offset] = value
return ic
```

```
#####
# Similarity metrics
#####
```

TODO: Add in the option to manually add a new root node; this will be
useful for verb similarity as there exist multiple verb taxonomies.

More information about the metrics is available at
<http://marimba.d.umn.edu/similarity/measures.html>

```
def path_similarity(synset1, synset2, verbose=False, simulate_root=True):
    return synset1.path_similarity(synset2, verbose, simulate_root) \[docs\]
```

```
def lch_similarity(synset1, synset2, verbose=False, simulate_root=True):
    return synset1.lch_similarity(synset2, verbose, simulate_root) \[docs\]
```

```
def wup_similarity(synset1, synset2, verbose=False, simulate_root=True):
    return synset1.wup_similarity(synset2, verbose, simulate_root) \[docs\]
```

```
def res_similarity(synset1, synset2, ic, verbose=False):
    return synset1.res_similarity(synset2, verbose) \[docs\]
```

```
def jcn_similarity(synset1, synset2, ic, verbose=False):
    return synset1.jcn_similarity(synset2, verbose) \[docs\]
```

```
def lin_similarity(synset1, synset2, ic, verbose=False):
    return synset1.lin_similarity(synset2, verbose) \[docs\]
```

```
path_similarity.__doc__ = Synset.path_similarity.__doc__
lch_similarity.__doc__ = Synset.lch_similarity.__doc__
wup_similarity.__doc__ = Synset.wup_similarity.__doc__
```

```
res_similarity.__doc__ = Synset.res_similarity.__doc__
jcn_similarity.__doc__ = Synset.jcn_similarity.__doc__
lin_similarity.__doc__ = Synset.lin_similarity.__doc__
```

```
def _lcs_ic(synset1, synset2, ic, verbose=False):
    """
    Get the information content of the Least common subsumer that has
    the highest information content value. If two nodes have no
    explicit common subsumer, assume that they share an artificial
    root node that is the hypernym of all explicit roots.

    :type synset1: Synset
    :param synset1: First input synset.
    :type synset2: Synset
    :param synset2: Second input synset. Must be the same part of
    speech as the first synset.
    :type ic: dict
    :param ic: an information content object (as returned by ``load_ic()``).
    :return: The information content of the two synsets and their most
    informative subsumer
    """
    if synset1._pos != synset2._pos:
        raise WordNetError(
            'Computing the least common subsumer requires '
            '%s and %s to have the same part of speech.' %
            (synset1, synset2)
        )

    ic1 = information_content(synset1, ic)
    ic2 = information_content(synset2, ic)
    subsumers = synset1.common_hyponyms(synset2)
    if len(subsumers) == 0:
        subsumer_ic = 0
    else:
        subsumer_ic = max(information_content(s, ic) for s in subsumers)

    if verbose:
        print("> LCS Subsumer by content:", subsumer_ic)

    return ic1, ic2, subsumer_ic
```

Utility functions

```
def information_content(synset, ic): \[docs\]
    try:
        icpos = ic[synset._pos]
    except KeyError:
        msg = 'Information content file has no entries for part-of-speech: %s'
        raise WordNetError(msg % synset._pos)

    counts = icpos[synset._offset]
    if counts == 0:
        return _INF
    else:
        return -math.log(counts / icpos[0])
```

get the part of speech (NOUN or VERB) from the information content record
 # (each identifier has a 'n' or 'v' suffix)

```
def _get_pos(field):
    if field[-1] == 'n':
        return NOUN
    elif field[-1] == 'v':
        return VERB
    else:
        msg = (
            "Unidentified part of speech in WordNet Information Content file "
```

```

        "for field %s" % field
    )
    raise ValueError(msg)

# unload corpus after tests
def teardown_module(module=None):
    from nltk.corpus import wordnet
    wordnet._unload()

#####
# Demo
#####

def demo():
    import nltk
    print('loading wordnet')
    wn = WordNetCorpusReader(nltk.data.find('corpora/wordnet'), None)
    print('done loading')
    S = wn.synset
    L = wn.lemma

    print('getting a synset for go')
    move_synset = S('go.v.21')
    print(move_synset.name(), move_synset.pos(), move_synset.lexname())
    print(move_synset.lemma_names())
    print(move_synset.definition())
    print(move_synset.examples())

    zap_n = ['zap.n.01']
    zap_v = ['zap.v.01', 'zap.v.02', 'nuke.v.01', 'microwave.v.01']

    def _get_synsets(synset_strings):
        return [S(synset) for synset in synset_strings]

    zap_n_synsets = _get_synsets(zap_n)
    zap_v_synsets = _get_synsets(zap_v)

    print(zap_n_synsets)
    print(zap_v_synsets)

    print("Navigations:")
    print(S('travel.v.01').hypernyms())
    print(S('travel.v.02').hypernyms())
    print(S('travel.v.03').hypernyms())

    print(L('zap.v.03.nuke').derivationally_related_forms())
    print(L('zap.v.03.atomize').derivationally_related_forms())
    print(L('zap.v.03.atomise').derivationally_related_forms())
    print(L('zap.v.03.zap').derivationally_related_forms())

    print(S('dog.n.01').member_holonyms())
    print(S('dog.n.01').part_meronyms())

    print(S('breakfast.n.1').hypernyms())
    print(S('meal.n.1').hyponyms())
    print(S('Austen.n.1').instance_hypernyms())
    print(S('composer.n.1').instance_hyponyms())

    print(S('faculty.n.2').member_meronyms())
    print(S('copilot.n.1').member_holonyms())

    print(S('table.n.2').part_meronyms())
    print(S('course.n.7').part_holonyms())

    print(S('water.n.1').substance_meronyms())
    print(S('gin.n.1').substance_holonyms())

    print(L('leader.n.1.leader').antonyms())

```

[\[docs\]](#)[\[docs\]](#)

```
print(L('increase.v.1.increase').antonyms())

print(S('snore.v.1').entailments())
print(S('heavy.a.1').similar_tos())
print(S('light.a.1').attributes())
print(S('heavy.a.1').attributes())

print(L('English.a.1.English').pertainyms())

print(S('person.n.01').root_hyponyms())
print(S('sail.v.01').root_hyponyms())
print(S('fall.v.12').root_hyponyms())

print(S('person.n.01').lowest_common_hyponyms(S('dog.n.01')))
print(S('woman.n.01').lowest_common_hyponyms(S('girlfriend.n.02')))

print(S('dog.n.01').path_similarity(S('cat.n.01')))
print(S('dog.n.01').lch_similarity(S('cat.n.01')))
print(S('dog.n.01').wup_similarity(S('cat.n.01')))

wnic = WordNetICCorpusReader(nltk.data.find('corpora/wordnet_ic'),
                             '.*\\.dat')

ic = wnic.ic('ic-brown.dat')
print(S('dog.n.01').jcn_similarity(S('cat.n.01'), ic))

ic = wnic.ic('ic-semcor.dat')
print(S('dog.n.01').lin_similarity(S('cat.n.01'), ic))

print(S('code.n.03').topic_domains())
print(S('pukka.a.01').region_domains())
print(S('freaky.a.01').usage_domains())

if __name__ == '__main__':
    demo()
```
